

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и компьютерной техники

Лабораторная работа №1

по дисциплине «Тестирование программного обеспечения»

Вариант: **330909**

Преподаватель:

Наумова Надежда Александровна

Выполнил:

Котляренко Кирилл Андреевич

Группа: Р3309

Санкт-Петербург, 2026

Описание задания

1. Для указанной функции провести модульное тестирование разложения функции в степенной ряд. Выбрать достаточное тестовое покрытие.
2. Провести модульное тестирование указанного алгоритма. Для этого выбрать характерные точки внутри алгоритма, и для предложенных самостоятельно наборов исходных данных записать последовательность попадания в характерные точки. Сравнить последовательность попадания с эталонной.
3. Сформировать доменную модель для заданного текста. Разработать тестовое покрытие для данной доменной модели

Вариант 330909

1. Функция $\arcsin(x)$
2. Программный модуль для обхода неориентированного графе методом поиска в ширину (<http://www.cs.usfca.edu/~galles/visualization/BFS.html>)
3. Описание предметной области:

После того, как толпа вновь разразилась ликующими криками, Артур обнаружил, что он скользит по воздуху к одному из величественных окон во втором этаже здания, перед которым стоял помост, с которого оратор обращался к народу.

Выполнение

Исходный код

<https://github.com/kkotlyarenko/Software-Testing-Labs/tree/main/lab1>

Задание 1: $\arcsin(x)$

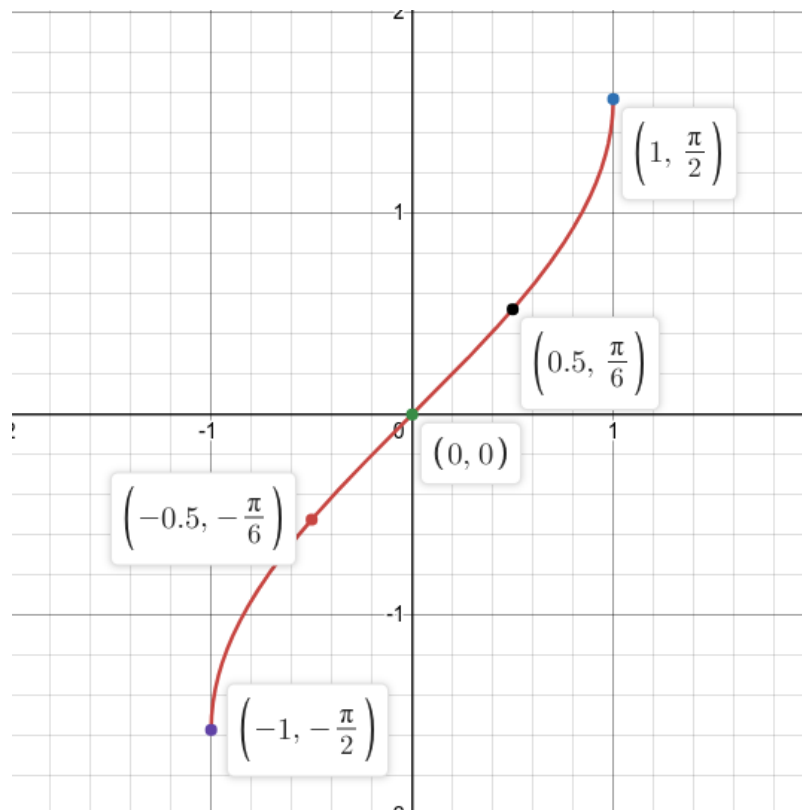


График функции $\arcsin(x)$ с табличными значениями в точках.

1. Выберем “интересные” для тестирования точки:

- $(x = 0)$
Базовая точка – ряд должен дать ноль
- Малые значения ($x \approx 0$)
При малых x выполняется приближение $\arcsin(x) \approx x$
- Табличные точные значения внутри $(-1; 1)$
Можно задать эталон без `java.lang.Math` используя известные углы
- Нечётность: пары (x) и $(-x)$
 $\arcsin(-x) = -\arcsin(x)$ свойство функции
- Границы области ($x = 1$) и ($x = -1$)
Крайние допустимые значения; у ряда медленная сходимость
- Проверка сходимости (влияние числа членов ряда)
При увеличении `terms` приближение должно улучшаться

Таблица тестовых данных для “интересных” точек:

x	$\arcsin(x)$	Почему
0	0	Базовая точка
1^{-8}	$\approx 1^{-8}$	Малые значения

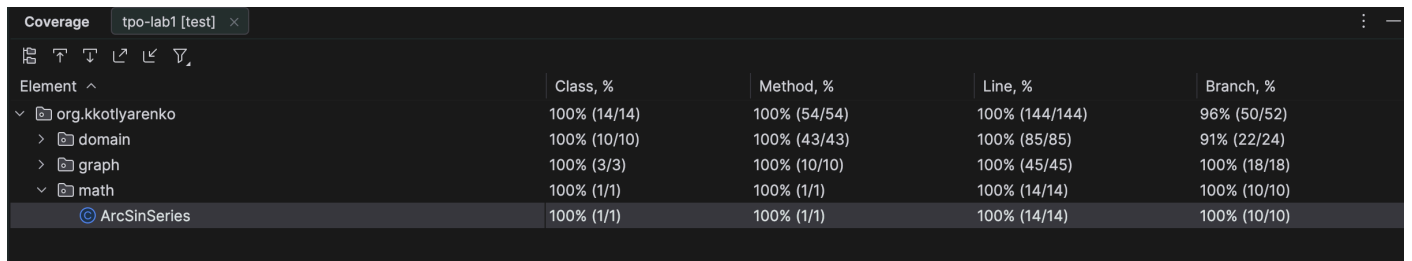
1^{-6}	$\approx 1^{-6}$	Малые значения
1^{-4}	$\approx 1^{-4}$	Малые значения
0.5	$\pi/6$	Табличное значение
0.7071067811865476	$\pi/4$	Табличное значение
0.8660254037844386	$\pi/3$	Табличное значение
-0.5	0.5	Нечётность
-0.7071067811865476	0.7071067811865476	Нечётность
-0.8660254037844386	0.8660254037844386	Нечётность
1	$\pi/2$	Граница области
-1	$-\pi/2$	Граница области

```

{ } math 48ms
  ArcSinSeriesTest 48ms
    testPositiveTableValues(double, double) Table values (positive): 2.0ms
      arcsin 1.0ms
      arcsin 0.0ms
      arcsin 0.0ms
      arcsin 1.0ms
    testNegativeTableValues(double, double) Table values (negative): 4.0ms
      arcsin 2.0ms
      arcsin 1.0ms
      arcsin 1.0ms
    testOddFunctionProperty(double) Odd function: arcsin(-x) = -arcsin(x): 5.0ms
      arcsin 1.0ms
      arcsin 1.0ms
      arcsin 1.0ms
      arcsin 0.0ms
      arcsin 1.0ms
      arcsin 1.0ms
    testBoundaryOne() Boundary value: arcsin(1) =  $\pi/2$ : 15ms
    testBoundaryMinusOne() Boundary value: arcsin(-1) =  $-\pi/2$ : 1.0ms
    testSmallValues(double) Small x: arcsin(x)  $\approx$  x: 0.0ms
      arcsin
      arcsin
      arcsin
    testConvergence(double) More terms produce better accuracy: 12ms
      Convergence at x=0.3 12ms
      Convergence at x=0.6
      Convergence at x=0.8
      Convergence at x=0.95
    testOutOfRange(double) IllegalArgumentException for |x| > 1: 7.0ms
      arcsin 4.0ms
      arcsin 0.0ms
      arcsin 1.0ms
      arcsin 1.0ms
      arcsin 1.0ms
    testInvalidTerms() IllegalArgumentException for terms <= 0: 2.0ms
    testSingleTerm() Single term: arcsin(x)  $\approx$  x: 0.0ms

```

Тесты успешно проходятся.



The screenshot shows the Coverage tool window in IntelliJ IDEA. The title bar indicates 'Coverage' and 'tpo-lab1 [test]'. The table below displays the coverage statistics for the project structure.

Element ^	Class, %	Method, %	Line, %	Branch, %
org.kkotlyarenko	100% (14/14)	100% (54/54)	100% (144/144)	96% (50/52)
domain	100% (10/10)	100% (43/43)	100% (85/85)	91% (22/24)
graph	100% (3/3)	100% (10/10)	100% (45/45)	100% (18/18)
math	100% (1/1)	100% (1/1)	100% (14/14)	100% (10/10)
ArcSinSeries	100% (1/1)	100% (1/1)	100% (14/14)	100% (10/10)

Итоговое тестовое покрытие.

Функция **arcsin(x)** корректно обрабатывает все тестовые случаи, включая граничные и специальные значения. Все результаты совпадают с эталонной реализацией **Math.asin**.

Функция правильно возвращает **NaN** для значений, выходящих за пределы допустимого диапазона, и сохраняет знак для нулевых значений.

Быстрая сходимость функции в окрестностях граничных значений не привела к ошибкам в вычислениях, что указывает на стабильность реализации.

Задание 2: BFS

Эталонная реализация по варианту: <https://www.cs.usfca.edu/~galles/visualization/BFS.html>

Поиск в ширину (BFS) — это алгоритм обхода неориентированного графа, который посещает вершины “слоями” по минимальному расстоянию от стартовой вершины и тем самым гарантирует корректное вычисление кратчайших расстояний (в рёбрах) в невзвешенном графе.

Для обеспечения корректности работы модуля обхода необходимо тщательно протестировать его основные аспекты:

- Обход графа из заданной стартовой вершины (порядок посещения).
- Вычисление расстояний до достижимых вершин.
- Формирование дерева поиска (родители).
- Обработка некорректных входных данных (границы, ошибки).

Цель тестирования — убедиться, что BFS корректно обходит граф (в пределах компоненты связности стартовой вершины), правильно вычисляет минимальные расстояния и сохраняет ожидаемые свойства результата при разных структурах графа.

Будем проверять нашу реализацию BFS и графа следующим образом:

- для каждого публичного метода модуля (создание графа / добавление ребра / получение соседей / запуск BFS – интерфейс совпадает с ожидаемым контрактом) попытаемся выделить основные сценарии работы метода (то есть возможные ветвления для метода)
- проверять приватные методы напрямую не будем; как “белый ящик” — будем проверять их корректность через вызов публичных методов на подобранных входных данных, которые гарантированно приводят к выполнению нужных ветвей (валидации вершин, обработка посещённости, работа очереди, обработка несвязности и т.д.).

Для тестирования BFS используется комбинация подходов:

1. Покрытие кода (Code Coverage):

Убедиться, что все ключевые ветви выполнены хотя бы один раз:

- успешный обход (очередь наполняется и опустошается);
- обход соседей с переходом по ветке “уже посещён / ещё не посещён”;
- обработка несвязного графа (частичное покрытие вершин);
- валидации: неверный `startVertex`, неверные вершины в `addEdge/getNeighbors`, неверный размер графа.

Особое внимание уделяется участкам, которые чаще всего дают ошибки:

- корректность очереди и порядка обработки соседей;
- корректность расчёта `distances` и заполнения `parents`.

2. Покрытие граничных значений (Boundary Value Analysis):

3. Тестирование на граничных случаях:

- Граф из одной вершины без рёбер (минимально допустимый размер).
- Несвязный граф: старт в одной компоненте → другие компоненты не должны появляться в результате.
- Стартовая вершина на границе диапазона (0 и $n-1$), а также ошибки: `startVertex < 0` и `startVertex ≥ n`.
- Создание графа с некорректным числом вершин (0, отрицательное).
- Добавление ребра с некорректными индексами вершин (вне диапазона).
- Получение соседей для некорректной вершины.

4. Покрытие свойств (Property-Based Testing):

- Проверка инвариантов результата BFS после обхода:
- Стартовая вершина всегда присутствует в результате, и `dist(start)=0`, `parent(start)=-1`.

- Для каждой достижимой вершины v расстояние $\text{dist}(v)$ минимально (в рёбрах) среди всех путей от старта.
 - Для каждой вершины $v \neq \text{start}$, если $\text{parent}(v)=p$, то $\text{dist}(v)=\text{dist}(p)+1$, а ребро (p, v) существует в графе.
 - Порядок посещения соответствует “слоям”: вершины с меньшим расстоянием не посещаются после вершин с большим расстоянием (с учётом того, что точный порядок внутри слоя зависит от порядка соседей).
5. Покрывание сценариев использования (Scenario-Based Testing):
- Тестирование типичных структур графов, на которых удобно проверять корректность обхода, расстояний и родителей:
 - Линейный граф (цепочка) — проверка последовательного роста расстояний и родителей.
 - “Ромб” — проверка обхода при наличии нескольких путей к вершине (важно, что расстояние остаётся минимальным).
 - “Звезда” — проверка массового добавления соседей одного слоя и корректных родителей от центра.
 - Полный граф K_4 — проверка, что все вершины (кроме старта) имеют расстояние 1.
 - Дерево — проверка слоёв и корректного построения дерева поиска.
 - Цикл — проверка, что обход не закичивается и правильно выбирает минимальные расстояния.
 - Старт из середины цепочки — проверка симметричного разрастания по слоям в обе стороны.

Как мы можем видеть, все работает корректно:

```

✓ { } graph 7.0ms
  ✓ BFSTest 7.0ms
    ✓ testLinearGraph() Linear graph: 0-1-2-3, start from 0: 0.0ms
    ✓ testDiamondGraph() Diamond graph: 0-1, 0-2, 1-3, 2-3, start from 0: 0.0ms
    ✓ testStarGraph() Star graph with center 0 and leaves 1-4: 0.0ms
    ✓ testDisconnectedGraph() Disconnected graph: single component traversal: 0.0ms
    ✓ testSingleVertex() Single vertex, no edges: 0.0ms
    ✓ testCompleteGraph() Complete graph K4, start from 0: 0.0ms
    ✓ testLinearGraphFromMiddle() Linear graph, start from the middle: 0.0ms
    ✓ testBinaryTree() Binary tree, start from root: 0.0ms
    ✓ testCycleGraph() Cycle of 4 vertices, start from 0: 2.0ms
    ✓ testInvalidStartNegative() Invalid start vertex (negative): 1.0ms
    ✓ testInvalidStartTooLarge() Invalid start vertex ( $\geq n$ ): 1.0ms
    ✓ testInvalidGraphCreation() Graph creation with invalid vertex count: 1.0ms
    ✓ testInvalidEdge() Edge creation with invalid vertex: 2.0ms
    ✓ testGetNeighborsInvalid() getNeighbors for invalid vertex: 0.0ms
    ✓ testGetVertexCount() getVertexCount returns expected value: 0.0ms

```

Тесты успешно проходятся.

graph	100% (3/3)	100% (10/10)	100% (45/45)	100% (18/18)
BFS	100% (2/2)	100% (5/5)	100% (29/29)	100% (10/10)
Graph	100% (1/1)	100% (5/5)	100% (16/16)	100% (8/8)

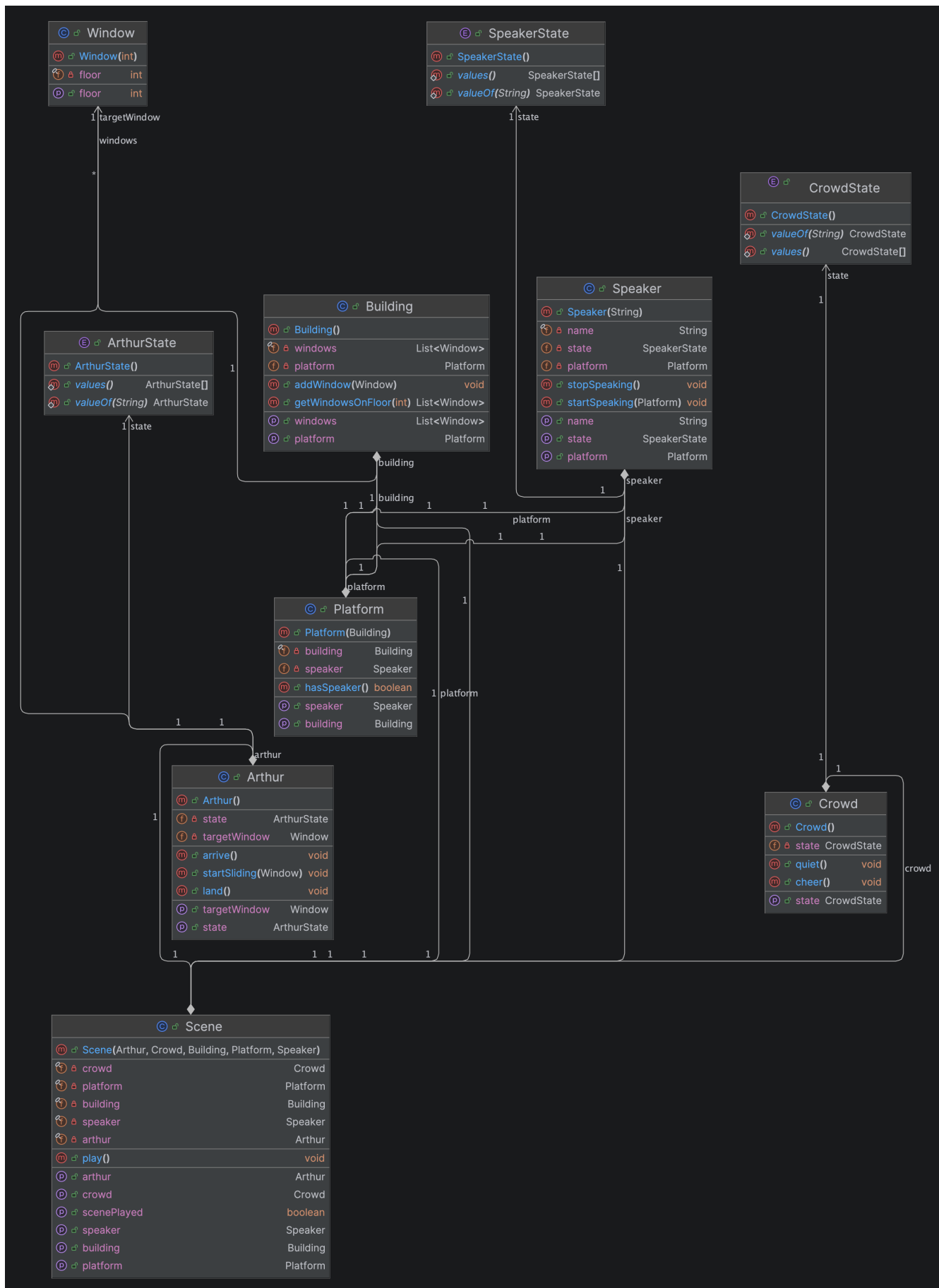
Итоговое тестовое покрытие методов класса RedBlackTree.

Результаты тестирования:

- BFS реализован корректно и проходит все тесты.
- Код покрыт модульными тестами.

Задание 3: Описание предметной области

Для начала, спроектируем нашу доменную область в соответствии с заданным текстом.



Спроектированная UML диаграмма предметной области.

Предметная область включает в себя:

- Персонажи (Артур, Толпа, Оратор).
- Объекты окружения (Здание, окно, помост).
- Состояния
 - Артур (STANDING -> SLIDING -> ARRIVED -> STANDING)
 - Толпа (QUIET -> CHEERING)
 - Оратор (IDLE -> SPEAKING)
- Действия
 - Артур: «начал скольжение к окну», «прибыть», «приземлиться»
 - Толпа: «ликовать» «успокаиваться»
 - Оратор: начал речь, остановил речь
 - Помост: содержит/не содержит оратора
- Сценарии (последовательности действий).

Далее, в соответствии с TDD напомним тесты по спроектированной модели.

Цель тестирования — убедиться, что:

- Все операции над сущностями выполняются корректно
- Состояния персонажей и объектов изменяются строго в соответствии с правилами переходов
- Связи между объектами корректно устанавливаются и поддерживаются
- Сценарий выполняется последовательно и приводит систему к ожидаемому набору состояний

Составим следующие тесты:

Класс: **Arthur**

- *initialState*: Проверяет корректность создания Артура (начальное состояние STANDING, целевое окно отсутствует).
- *startSliding*: Проверяет переход STANDING → SLIDING и установку целевого окна.
- *arrive*: Проверяет переход SLIDING → ARRIVED.
- *land*: Проверяет переход ARRIVED → STANDING и сброс целевого окна.
- *cannotSlideFromSliding*: Проверяет, что повторный старт скольжения вне STANDING запрещён (ошибка состояния).
- *cannotArriveFromStanding*: Проверяет, что “прибыть” вне SLIDING запрещено (ошибка состояния).
- *cannotSlideToNullWindow*: Проверяет валидацию входных данных (нельзя скользить к null окну).
- *fullCycle*: Проверяет полный корректный цикл состояний STANDING → SLIDING → ARRIVED → STANDING.

Класс: **Crowd**

- *initialState*: Проверяет начальное состояние толпы (QUIET).
- *cheer*: Проверяет переход QUIET → CHEERING.
- *quietDown*: Проверяет переход CHEERING → QUIET.
- *cheerTwice*: Проверяет устойчивость к повторному действию (повторный cheer() не ломает состояние и оставляет CHEERING).

Класс: **Speaker**

- *initialState*: Проверяет корректность создания оратора (имя, начальное состояние IDLE, отсутствие помоста).
- *startSpeaking*: Проверяет переход IDLE → SPEAKING, установку помоста у оратора и установку оратора на помосте.
- *stopSpeaking*: Проверяет переход SPEAKING → IDLE.

- *cannotSpeakWithoutPlatform*: Проверяет валидацию входных данных (нельзя начать речь без помоста).

Класс: **Window**

- *validWindow*: Проверяет корректное создание окна с валидным этажом.
- *invalidFloor*: Проверяет, что этаж ≤ 0 запрещён (валидация данных).

Класс: **Building**

- *addAndGetWindows*: Проверяет добавление окон и корректное хранение списка окон.
- *getWindowsOnFloor*: Проверяет фильтрацию окон по этажу (в т.ч. отсутствие окон на несуществующем этаже).
- *platformAttachment*: Проверяет, что при создании помоста для здания связь “здание $\leftrightarrow$ помост” устанавливается корректно.

Класс: **Platform**

- *creation*: Проверяет корректное создание помоста и связь с зданием, а также начальное отсутствие оратора на помосте.
- *nullBuilding*: Проверяет валидацию входных данных (нельзя создать помост без здания).
- *speakerOnPlatform*: Проверяет, что после начала речи оратора помост “содержит оратора” и это отражается в состоянии/ссылках.

Класс: **Scene** (сценарий предметной области)

- *initialState*: Проверяет начальные состояния всех участников сценария и флаг “сценарий не проигран”.
- *getters*: Проверяет целостность ссылок (что сцена работает с теми же объектами, что были переданы).
- *playFullScenario*: Проверяет выполнение полного сценария: оратор начинает речь на помосте, толпа ликует, Артур начинает скольжение к окну 2-го этажа, сценарий помечается как выполненный.
- *arthurArrives*: Проверяет, что после проигрывания сценария допустим следующий шаг — Артур может “прибыть” (переход к ARRIVED).
- *platformBuilding*: Проверяет инвариант связи “помост принадлежит зданию” в составе сцены.
- *speakerOnPlatformAfterPlay*: Проверяет, что после play() оратор действительно находится на помосте (связь установлена, hasSpeaker=true).

```

✓ { } domain 9.0ms
  ✓ DomainModelTest 9.0ms
    ✓ ArthurTests Arthur: 4.0ms
      ✓ initialState() Initial state is STANDING: 0.0ms
      ✓ startSliding() Transition STANDING → SLIDING: 1.0ms
      ✓ arrive() Transition SLIDING → ARRIVED: 1.0ms
      ✓ land() Transition ARRIVED → STANDING (land): 0.0ms
      ✓ cannotSlideFromSliding() Cannot start sliding outside STANDING: 1.0ms
      ✓ cannotArriveFromStanding() Cannot arrive outside SLIDING: 1.0ms
      ✓ cannotSlideToNullWindow() Cannot slide to null window: 0.0ms
      ✓ fullCycle() Full cycle: STANDING → SLIDING → ARRIVED → STANDING: 0.0ms
    ✓ CrowdTests Crowd: 1.0ms
      ✓ initialState() Initial state is QUIET: 0.0ms
      ✓ cheer() Transition QUIET → CHEERING: 0.0ms
      ✓ quietDown() Transition CHEERING → QUIET: 0.0ms
      ✓ cheerTwice() Repeated cheer remains CHEERING: 1.0ms
    ✓ SpeakerTests Speaker: 0.0ms
      ✓ initialState() Initial state is IDLE: 0.0ms
      ✓ startSpeaking() Start speaking: IDLE → SPEAKING: 0.0ms
      ✓ stopSpeaking() Stop speaking: SPEAKING → IDLE: 0.0ms
      ✓ cannotSpeakWithoutPlatform() Cannot speak without platform (null): 0.0ms
    ✓ WindowTests Window: 0.0ms
      ✓ validWindow() Window creation with valid floor: 0.0ms
      ✓ invalidFloor() Cannot create window with floor <= 0: 0.0ms
    ✓ BuildingTests Building: 1.0ms
      ✓ addAndGetWindows() Add and retrieve windows: 0.0ms
      ✓ getWindowsOnFloor() Filter windows by floor: 1.0ms
      ✓ platformAttachment() Attach platform to building: 0.0ms
    ✓ PlatformTests Platform: 2.0ms
      ✓ creation() Create platform for building: 0.0ms
      ✓ nullBuilding() Cannot create platform with null building: 1.0ms
      ✓ speakerOnPlatform() Speaker on platform: 1.0ms
    ✓ SceneTests Scene — full scenario: 1.0ms
      ✓ initialState() Initial scene state: 0.0ms
      ✓ getters() Scene getters: 0.0ms
      ✓ playFullScenario() play() executes full text scenario: 0.0ms
      ✓ arthurArrives() Arthur can arrive after play(): 0.0ms
      ✓ platformBuilding() Relation: platform belongs to building: 0.0ms
      ✓ speakerOnPlatformAfterPlay() Relation: speaker is on platform after play(): 1.0ms

```

Тесты успешно проходятся.

org.kkotlyarenko	100% (14/14)	100% (54/54)	100% (144/144)	96% (50/52)
domain	100% (10/10)	100% (43/43)	100% (85/85)	91% (22/24)
model	100% (7/7)	100% (37/37)	100% (75/75)	91% (22/24)
Arthur	100% (1/1)	100% (6/6)	100% (15/15)	100% (6/6)
Building	100% (1/1)	100% (6/6)	100% (11/11)	100% (4/4)
Crowd	100% (1/1)	100% (4/4)	100% (5/5)	100% (0/0)
Platform	100% (1/1)	100% (5/5)	100% (9/9)	100% (4/4)
Scene	100% (1/1)	100% (8/8)	100% (18/18)	66% (4/6)
Speaker	100% (1/1)	100% (6/6)	100% (12/12)	100% (2/2)
Window	100% (1/1)	100% (2/2)	100% (5/5)	100% (2/2)
state	100% (3/3)	100% (6/6)	100% (10/10)	100% (0/0)
ArthurState	100% (1/1)	100% (2/2)	100% (4/4)	100% (0/0)
CrowdState	100% (1/1)	100% (2/2)	100% (3/3)	100% (0/0)
SpeakerState	100% (1/1)	100% (2/2)	100% (3/3)	100% (0/0)

Итоговое тестовое покрытие.

Выводы

Во время выполнения лабораторной работы я углубил свои знания в JUnit5, научился писать юнит-тесты, использовать параметризированные тесты и тесты на проверку составленной объектной модели.